Treehouse tETH RedemptionV3 Upgrade

Aug 12, 2026

Table of Contents

Summary 2

Overview 3

Issues 4

[WP-M1] Redemption terms are read live at finalize — the 1-day ops role can retroactively confiscate or freeze in-flight redemptions 4

[WP-M2] The 1-day ops timelock can steal every NFT approved to the exemption gate — contradicting the deployment's written risk acceptance 9

[WP-L3] V3 accepts irreversible deposits for ~5 days before it is able to service them 13

[WP-L4] `Rescuable` can withdraw the tETH backing every pending redemption 17

[WP-L5] The Fastlane earmark uses the live share price, under-reserving across a dip-and-recover 19

[WP-L6] Blacklist asymmetry: escrowed value escapes the blacklist, while blacklisting V3 freezes every queued exit 22

[WP-I7] Ops timelock proposer/executor set is a placeholder (admin multisig) — deploying as-is collapses the two-tier governance model 24

[WP-I8] The fast-pause EOA can also un-pause, so a leaked pause key can undo an emergency stop 26

[WP-I9] Fastlane checks the liquidity reservation before the 4626 redeem, allowing a 1-wei violation 28

[WP-G10] Fastlane issues an unconditional treasury payout even when the fee rounds to zero 30

[WP-G11] PRECISION should not be a mutable storage variable; declare it constant can save gas 32

[WP-N12] setPause → setPathPause rename leaves off-chain pause tooling calling the old selector 34

Appendix 36

Disclaimer 37

Summary

This report has been prepared for Treehouse smart contract, to discover issues and vulnerabilities in the source code of their Smart Contract as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

Overview

Project Summary

Project Name	**Treehouse**
Codebase	**https://github.com/treehouse-gaia/tETH-protocol**
Commit	**aa9b25002c40120579f0c3d7e7c65df0b944b7d9**
Language	**Solidity**

Audit Summary

Delivery Date	**Aug 12, 2026**
Audit Methodology	**Static Analysis, Manual Review**
Total Isssues	**12**

[WP-M1] Redemption terms are read live at finalize — the 1-day ops role can retroactively confiscate or freeze in-flight redemptions

Medium

Issue Description

`TreehouseRedemptionV3.finalizeRedeem` computes the payout from `redemptionFee` and `treasuryFee` and gates finalization on `waitingPeriod`, all read from LIVE storage at finalize time. The `RedemptionInfo` struct snapshots only `startTime/shares/assets/baseRate` — never the fee or the waiting period. All three setters (`setRedemptionFee`, `setTreasuryFee`, `setWaitingPeriod`) are `onlyOwnerOrOps`, callable by the new 1-day ops timelock; the combined fee cap is 100% (`PRECISION`), and `treasuryFee` routes value to the owner-set `treasury` address. There is no way to cancel a pending redemption — once `redeem()` / `redeemWithNft()` escrows the tAsset, the only exit is `finalizeRedeem` after the wait.

Invariant violated: the economic terms a user commits to at redeem time are not the terms enforced at finalize.

Worst case: an ops proposal maturing in 1 day (shorter than the 7-day `waitingPeriod`, so committed users cannot escape) can set `treasuryFee = 10000` → a finalizing user's entire `_gross` is redeemed to `treasury` and `_returnAmount == 0` (user receives nothing), or raise `waitingPeriod` toward `type(uint32).max` to freeze finalization indefinitely.

PoC

1. User calls `redeem(shares)` (or `redeemWithNft`), escrowing tAsset; a `RedemptionInfo` entry is stored with NO fee/wait snapshot.
2. Before the 7-day wait elapses, ops schedules `setTreasuryFee(10000)` (or `setWaitingPeriod(type(uint32).max)`), which matures in 1 day.
3. After the wait, the user calls `finalizeRedeem`; `_treasuryCut = _gross * 10000 / 10000 == _gross`, `_returnAmount = _gross - _holderFee - _treasuryCut` collapses to 0 after the holderFee is also 0 (or the whole gross goes to treasury), and the payout leaves to `treasury`. The user cannot avoid it (no cancel; ops delay < wait).

The `RedemptionInfo` struct — snapshots `startTime/shares/assets/baseRate` only, no fee/wait field:

```
39    struct RedemptionInfo {
40      uint64 startTime;
41      uint96 shares;
42      uint128 assets;
43      uint128 baseRate;
44    }
```

`finalizeRedeem` reads `waitingPeriod` , `redemptionFee` , and `treasuryFee` LIVE from storage, and routes `_treasuryCut` to `treasury` :

```
160   function finalizeRedeem(
161     uint _redeemIndex
162   ) external nonReentrant whenNotPaused validateRedeem(msg.sender, _redeemIndex) {
163     RedemptionInfo storage _redeem = redemptionInfo[msg.sender][_redeemIndex];
164
165     if (block.timestamp < _redeem.startTime + waitingPeriod) revert
    InWaitingPeriod();
166     uint _assets = IERC4626(TASSET).redeem(_redeem.shares, address(this),
    address(this));
167     redeeming[msg.sender] -= _redeem.shares;
168     totalRedeeming -= _redeem.shares;
169
170     address _underlying = VAULT.getUnderlying();
171
172     uint _gross = _getReturnAmount(_redeem.assets, _redeem.baseRate, _assets,
    _getBaseRate());
173     uint _holderFee = (_gross * redemptionFee) / PRECISION;
174     uint _treasuryCut = (_gross * treasuryFee) / PRECISION;
175     uint _returnAmount = _gross - _holderFee - _treasuryCut;
176
177     if (_gross > _redeem.assets) revert RedemptionError();
178
179     // amount actually leaving the vault: user payout + treasury cut (holderFee
    stays as donated IAU)
180     uint _outflow = _returnAmount + _treasuryCut;
181     if (IERC20(_underlying).balanceOf(address(VAULT)) < _outflow) revert
    InsufficientFundsInVault();
182
```

```
183       IInternalAccountingUnit(IAU).burn(_outflow);
184       REDEMPTION_CONTROLLER.redeem(_returnAmount, msg.sender);
185       if (_treasuryCut > 0) {
186         REDEMPTION_CONTROLLER.redeem(_treasuryCut, treasury);
187       }
      @@ 188,197 @@
198       emit RedeemFinalized(msg.sender, _returnAmount, _holderFee + _treasuryCut);
```

The three ops-gated setters — all `onlyOwnerOrOps` (i.e. the 1-day ops timelock), combined cap is only `PRECISION` (100%):

```
208   function setWaitingPeriod(uint32 _newWaitingPeriod) external onlyOwnerOrOps {
209     emit WaitingPeriodUpdated(_newWaitingPeriod, waitingPeriod);
210     waitingPeriod = _newWaitingPeriod;
211   }
```

```
229   function setRedemptionFee(uint32 _newFee) external onlyOwnerOrOps {
230     if (_newFee + treasuryFee > PRECISION) revert FeeExceeded();
231     emit RedemptionFeeUpdated(_newFee, redemptionFee);
232     redemptionFee = _newFee;
233   }
```

```
240   function setTreasuryFee(uint32 _newFee) external onlyOwnerOrOps {
241     if (redemptionFee + _newFee > PRECISION) revert FeeExceeded();
242     emit TreasuryFeeUpdated(_newFee, treasuryFee);
243     treasuryFee = _newFee;
244   }
```

The deploy script explicitly documents this as ACCEPTED RISK:

```
76    *   ⚠ ACCEPTED RISK (2026-07-20, extended 2026-07-23; no hard caps): fees (≤100%
77    *   combined), waitingPeriod (≤uint32.max), and the exemption-gate pointer are
      settable
78    *   at 1-day ops speed; fees/waitingPeriod are read LIVE at finalize. A rogue ops
79    *   proposal can zero/freeze in-flight redeemers, or repoint the gate (bounded:
      NFT-path
```

```
80    *   DoS or free floor bypass — no theft; user NFT approvals are per-gate). The
81    *   ops-governance set below is the primary protection.
```

Recommendation

Snapshot the effective `redemptionFee` / `treasuryFee` (and ideally `waitingPeriod`) into `RedemptionInfo` at redeem time and read the snapshot at finalize; OR cap `treasuryFee` / `redemptionFee` at a low maximum; OR route fee/waiting-period changes through the 5-day main timelock only (keep ops for the min-floor/gate kill-switch).

Corrected-code sketch — snapshot the terms into `RedemptionInfo` at `_processRedeem` and consume them in `finalizeRedeem`:

```
struct RedemptionInfo {
  uint64 startTime;
  uint96 shares;
  uint128 assets;
  uint128 baseRate;
  uint32 redemptionFee; // snapshot at redeem time
  uint32 treasuryFee;   // snapshot at redeem time
  // optionally: uint32 waitingPeriod;
}

function _processRedeem(uint96 _shares, uint128 _assets) internal {
  IERC20(TASSET).safeTransferFrom(msg.sender, address(this), _shares);

  redemptionInfo[msg.sender].push(
    RedemptionInfo({
      startTime: block.timestamp.toUint64(),
      assets: _assets,
      shares: _shares,
      baseRate: _getBaseRate().toUint128(),
      redemptionFee: redemptionFee, // lock in the terms the user commits to
      treasuryFee: treasuryFee
    })
  );
  // ...
}

function finalizeRedeem(uint _redeemIndex) external /* ... */ {
```

```
    RedemptionInfo storage _redeem = redemptionInfo[msg.sender][_redeemIndex];
    // ...
    uint _holderFee = (_gross * _redeem.redemptionFee) / PRECISION; // read the snapshot
    uint _treasuryCut = (_gross * _redeem.treasuryFee) / PRECISION;
    // ...
  }
```

Status

ⓘ Acknowledged

[WP-M2] The 1-day ops timelock can steal every NFT approved to the exemption gate — contradicting the deployment's written risk acceptance

Medium

Issue Description

`NftExemptionGate` is constructor-owned by the NEW 1-day ops `TimelockController`, and `consumeTicket` pulls an ERC721 **from an arbitrary** `_user`, relying only on that user's standing approval to the gate. Two owner-only setters combine into a theft path the deployment's risk model does not account for:

- `addConsumer(address)` authorizes any address to call `consumeTicket`.
- `setNftTreasury(address)` redirects where every surrendered NFT lands.

A single ops batch — `addConsumer(attacker)` + `setNftTreasury(attacker)` — lets the attacker drain every accepted-collection NFT held by any user who has approved the gate, at 1-day speed. Users are steered toward blanket approval rather than per-token approval: the deploy script's own smoke test calls `setApprovalForAll(gate, true)`, so in practice each user exposes their entire Squirrel Council holding, not one token.

Why this is more than ordinary admin risk. The deploy script's ACCEPTED RISK block states the exemption-gate exposure is *"bounded: NFT-path DoS or free floor bypass — no theft; user NFT approvals are per-gate."* The "per-gate" argument correctly bounds `TreehouseRedemptionV3.setExemptionGate` repointing — a freshly deployed gate holds no approvals — but it does not bound **the existing gate's own owner**. The risk was accepted on an incorrect model.

For contrast, the same "no theft" claim **does** hold on the fee path: `setRedemptionFee` / `setTreasuryFee` are capped at a combined 100%, but `setTreasury` is `onlyOwner` (5-day main timelock), so ops acting alone can zero a payout yet cannot route it anywhere it controls. The NFT path has no such separation — `setNftTreasury` is ops-reachable.

Aggravating: the ops proposer/executor set is still a `TODO(team)` placeholder equal to `ADMIN_MS`, so as written the two-tier model collapses to one signer set holding a 1-day path to NFT theft.

Impact: theft of all NFTs approved to the gate, by a role the team has documented as incapable of theft.

Vulnerable Code

```
61    function consumeTicket(address _user, address _collection, uint _tokenId)
      external {
62      if (!_consumers.contains(msg.sender)) revert Unauthorized();
63      if (!_acceptedCollections.contains(_collection)) revert
        CollectionNotAccepted();
64
65      IERC721(_collection).safeTransferFrom(_user, nftTreasury, _tokenId);
66
67      emit TicketConsumed(_user, _collection, _tokenId, msg.sender);
68    }
```

```
98    /**
99     * @notice Authorize a redemption contract to consume tickets
100    * @param _consumer redemption contract to authorize
101    */
102   function addConsumer(address _consumer) external onlyOwner {
103     if (_consumer == address(0)) revert ZeroAddress();
104     if (!_consumers.add(_consumer)) revert ConsumerUpdateFailed();
105     emit ConsumerAdded(_consumer);
106   }
```

```
117   /**
118    * @notice Set the treasury receiving surrendered NFTs
119    * @dev the treasury must never recirculate surrendered NFTs — every NFT
      returned to
120    *      circulation is a fresh exemption ticket
121    * @param _newTreasury new NFT treasury address
122    */
123   function setNftTreasury(address _newTreasury) external onlyOwner {
124     if (_newTreasury == address(0)) revert ZeroAddress();
125     emit NftTreasuryUpdated(_newTreasury, nftTreasury);
126     nftTreasury = _newTreasury;
127   }
```

The gate is owned by the ops timelock, so both setters above are ops-reachable:

```
161       // 3. NFT exemption gate — ctor-owned by the ops timelock; addConsumer(V3)
          rides
162       //    the ops-timelock batch (avoids a deployer own/transfer/accept dance)
163       address[] memory _collections = new address[](1);
164       _collections[0] = SQUIRREL_COUNCIL;
165       NftExemptionGate _gate = new NftExemptionGate(address(_opsTimelock),
          NFT_TREASURY, _collections);
```

The risk acceptance this finding contradicts:

```
74    *     rotation, escape hatch on all ops setters) — and keeps owning
75    *     V2 / RedemptionController / IAU.
76    *   ⚠ ACCEPTED RISK (2026-07-20, extended 2026-07-23; no hard caps): fees (≤100%
77    *   combined), waitingPeriod (≤uint32.max), and the exemption-gate pointer are
      settable
78    *   at 1-day ops speed; fees/waitingPeriod are read LIVE at finalize. A rogue ops
79    *   proposal can zero/freeze in-flight redeemers, or repoint the gate (bounded:
      NFT-path
80    *   DoS or free floor bypass — no theft; user NFT approvals are per-gate). The
81    *   ops-governance set below is the primary protection.
```

Recommendation

Consider removing the arbitrary-victim parameter entirely. The root cause is the `_user` argument on `consumeTicket` — it lets any authorized consumer name a victim. Nothing that merely slows down consumer authorization removes that. Instead, have the redemption contract pull the NFT itself, with `from` hardcoded to the caller, and demote the gate to a policy oracle that holds no transfer authority:

```
// TreehouseRedemptionV3
function redeemWithNft(uint96 _shares, address _collection, uint _tokenId) external
nonReentrant whenNotPaused {
  if (address(exemptionGate) == address(0)) revert ZeroAddress();

  uint128 _assets = IERC4626(TASSET).previewRedeem(_shares).toUint128();
  if (_assets == 0) revert MinimumNotMet();
```

```
    if (!exemptionGate.isAcceptedCollection(_collection)) revert CollectionNotAccepted();

    // `from` is msg.sender, not a parameter — no caller can name a victim
    IERC721(_collection).safeTransferFrom(msg.sender, NFT_TREASURY, _tokenId);

    _processRedeem(_shares, _assets);
  }
```

With this shape:

- `addConsumer` / `removeConsumer` can be **deleted outright** — the gate no longer moves tokens, so a consumer allow-list has nothing to authorize.
- User approvals sit on `TreehouseRedemptionV3`, which is a plain non-upgradeable contract. Its inherited `Rescuable` exposes only `rescueERC20` and `rescueETH` — there is no ERC721 path — and under this design V3 never custodies an NFT, since the transfer goes straight from the user to the treasury. No admin function can reach a user's NFT.
- The residual admin power is the *destination*, and only prospectively: redirecting it affects users who call `redeemWithNft` after the change, and those users are receiving their redemption in exchange. It cannot retroactively drain standing approvals. Close it fully by making the treasury `immutable` on V3, or at minimum `onlyOwner` (5-day) rather than reading it from an ops-owned gate.

Status

✓ Fixed

[WP-L3] V3 accepts irreversible deposits for ~5 days before it is able to service them

Low

Issue Description

The deployment plan treats activation as the moment the main 5-day timelock batch executes, but nothing prevents users from entering `TreehouseRedemptionV3` from the instant it is deployed. The permissions required at the two ends of the flow are asymmetric:

- **Entering the queue** needs only a tETH `transferFrom` into V3. It requires neither IAU minter status nor controller whitelisting.
- **Leaving the queue** needs both: `finalizeRedeem` calls `IERC4626(TASSET).redeem(...)`, and `TAsset._withdraw` reverts unless the caller is an IAU minter; `REDEMPTION_CONTROLLER.redeem` likewise reverts for a non-whitelisted caller.

V3 is deployed unpaused and both grants land only in the `t0+5d` batch. So between deployment and activation — and, once the ops batch lands at `t0+1d`, including the NFT path — a user can escrow tETH and irreversibly surrender an NFT into a contract where every `finalizeRedeem` reverts. **V3 has no cancel or emergency-withdraw function**, so those users are stuck until activation; if the batch is delayed or cancelled the tETH is stranded permanently and the NFT is spent for nothing. The rescue path is closed too, since `run()` never appoints a rescuer.

A secondary effect compounds it: during the same window the still-live legacy `TreehouseFastlane` earmarks only V2, so it can pay out wstETH that early V3 claims will later need.

Impact: up to the full value of any pre-activation V3 entries becomes illiquid with no user-side recovery path, and NFT tickets can be burned against claims that cannot settle.

Vulnerable Code

Entering the queue needs no protocol permission — only a token transfer:

```
136   function _processRedeem(uint96 _shares, uint128 _assets) internal {
137     IERC20(TASSET).safeTransferFrom(msg.sender, address(this), _shares);
138
139     redemptionInfo[msg.sender].push(
140       RedemptionInfo({
141         startTime: block.timestamp.toUint64(),
142         assets: _assets,
143         shares: _shares,
144         baseRate: _getBaseRate().toUint128()
145       })
146     );
147
148     unchecked {
149       redeeming[msg.sender] += _shares;
150       totalRedeeming += _shares;
151     }
152
153     emit Redeemed(msg.sender, _shares, _assets);
154   }
```

Leaving it does, via the 4626 redeem inside `finalizeRedeem`:

```
160   function finalizeRedeem(
161     uint _redeemIndex
162   ) external nonReentrant whenNotPaused validateRedeem(msg.sender, _redeemIndex) {
163     RedemptionInfo storage _redeem = redemptionInfo[msg.sender][_redeemIndex];
164
165     if (block.timestamp < _redeem.startTime + waitingPeriod) revert
      InWaitingPeriod();
166     uint _assets = IERC4626(TASSET).redeem(_redeem.shares, address(this),
      address(this));
167     redeeming[msg.sender] -= _redeem.shares;
168     totalRedeeming -= _redeem.shares;
```

```
69    /**
70     * @dev Only callable by IAU minters
71     */
72    function _withdraw(
73      address caller,
```

```
74      address receiver,
75      address owner,
76      uint256 assets,
77      uint256 shares
78    ) internal virtual override {
79      if (!IInternalAccountingUnit(asset()).isMinter(caller)) revert Unauthorized();
80      super._withdraw(caller, receiver, owner, assets, shares);
81    }
```

The plan assumes nothing is live until the main batch, but never enforces it:

```
83    * Sequencing (old contracts are Timelock-owned — nothing live changes at deploy
      time):
84    *   t0      run() — deploy + config + transferOwnership; emits Safe Transaction
      Builder
85    *           JSONs into safe-txs/ and the address book into deployments/.
86    *   t0      admin Safe imports + signs redemption-update-schedule.json (main
      Timelock)
87    *           and redemption-ops-schedule.json (ops timelock).
88    *   t0+1d   execute ops batch: gate.addConsumer(V3).
89    *   t0+5d   execute main batch (atomic): accept ownership of V3/FastlaneV2, grant
      IAU
90    *           minters, whitelist both on the controller, cut V2 inflow
91    *           (setMinRedeem(uint96.max) — finalize stays open; V2 stays whitelisted
      and an
92    *           IAU minter FOREVER as the legacy exit), decommission the old Fastlane
93    *           (removeRedemption + removeMinter).
94    *   then    FE flips to V3 / FastlaneV2. V2 book drains within its 7-day waiting
      period.
```

Recommendation

Deploy paused and unpause as the final step of the main batch. `setPathPause` is owner-or-pauser and the timelock accepts ownership at index 0 of the same batch, so this composes atomically:

```
// in run(), before transferOwnership:
_v3.setPathPause(true);
_fastlaneV2.setPathPause(true);
```

```
// append to _mainBatch (arrays resized to 11):
_targets[9]  = _v3;
_payloads[9] = abi.encodeCall(PathPausable.setPathPause, (false));

_targets[10]  = _fastlaneV2;
_payloads[10] = abi.encodeCall(PathPausable.setPathPause, (false));
```

Independently, consider adding a `cancelRedeem(uint _redeemIndex)` that returns the escrowed tETH before the waiting period elapses. Its absence is what turns every sequencing slip into a permanent user loss rather than a delay.

Status

✓ Fixed

[WP-L4] `Rescuable` can withdraw the tETH backing every pending redemption

`Low`

Issue Description

`TreehouseRedemptionV3` inherits Circle's `Rescuable`, whose `rescueERC20` accepts **any** ERC20 — including `TASSET`, the token V3 holds in escrow for pending redeemers. A rescuer can empty that escrow while `totalRedeeming` and `redeeming[user]` remain untouched, so:

1. pending users lose the collateral backing their claims and `finalizeRedeem` reverts for want of shares; and
2. the stale `totalRedeeming` keeps earmarking `TreehouseFastlaneV2` liquidity for claims that can never settle, needlessly throttling the atomic path.

Reported as low rather than medium because `_rescuer` defaults to `address(0)` and `RedemptionUpdate.run()` never calls `updateRescuer`, so the path is disabled as deployed, and appointing a rescuer is `onlyOwner` (5-day main timelock). It is also inherited convention — V2 and the old Fastlane carry the identical pattern. It is raised here because V3's escrow is materially larger than V2's: the floor drops from 250 to 50, and `redeemWithNft` removes it entirely.

Impact: conditional loss of the entire queued tETH balance if a rescuer is ever appointed, plus a permanently desynchronised earmark.

```
50  contract TreehouseRedemptionV3 is ITreehouseRedemptionV3, Ownable2Step,
    ReentrancyGuard, PathPausable, OpsManaged, Rescuable {
```

```
51    /**
52     * @notice Rescue ERC20 tokens locked up in this contract.
53     * @param tokenContract ERC20 token contract address
54     * @param to        Recipient address
55     * @param amount    Amount to withdraw
56     */
57    function rescueERC20(IERC20 tokenContract, address to, uint256 amount) external
      onlyRescuer {
```

```
58        tokenContract.safeTransfer(to, amount);
59      }
```

Recommendation

Override the rescue entry point so it can never reach user escrow. An operational rescuer should be able to recover stray tokens without being able to touch the asset the contract custodies:

```
// TreehouseRedemptionV3 (and TreehouseFastlaneV2, for symmetry)
error CannotRescueEscrow();

function rescueERC20(IERC20 tokenContract, address to, uint256 amount) external override
onlyRescuer {
  if (address(tokenContract) == TASSET) revert CannotRescueEscrow();
  super.rescueERC20(tokenContract, to, amount);
}
```

This requires making `Rescuable.rescueERC20` `virtual` .

Status

ⓘ Acknowledged

[WP-L5] The Fastlane earmark uses the live share price, under-reserving across a dip-and-recover

Low

Issue Description

`TreehouseFastlaneV2.getRedeemableAmount()` reserves vault liquidity for pending delayed claims as `convertToAssets( totalRedeeming())` , evaluated at the **current** tETH share price. But a V3 claim's payout is capped at `_redeem.assets` — its value at *redeem* time — enforced by the `RedemptionError` check in `finalizeRedeem` .

The two diverge when the share price falls and later recovers. During the dip the earmark is understated relative to what the queue will eventually draw, so the atomic path is permitted to release wstETH that the recovered claims need. Delayed redeemers then revert with `InsufficientFundsInVault` until strategy liquidity is returned to the vault.

The premise is reachable rather than theoretical: `TreehouseAccounting.mark(BURN, ...)` calls `IInternalAccountingUnit.burnFrom(TASSET, ...)` , which reduces `totalAssets` and therefore the share price. Loss socialisation is exactly the scenario in which the queue is most crowded.

Impact: availability, not value loss — the claim still settles once the vault is replenished, and the atomic redeemer receives only the fair value of their own shares.

Vulnerable Code

The reservation is computed at the live rate:

```
171    /**
172     * @notice Get amount of underlying that can be atomically redeemed from vault
173     * @dev earmarks liquidity for the summed pending claims across every registered
174     *      delayed-redemption contract, so the atomic path never spends wstETH
    already
175     *      promised to a pending delayed redemption
176     * @return _totalRedeemable amount that can be atomically redeemed
177     */
178    function getRedeemableAmount() public view returns (uint _totalRedeemable) {
179      uint _underlyingInVault = IERC20(UNDERLYING).balanceOf(address(VAULT));
180
```

```
181       // sum shares first (all paths escrow the same TASSET), then convert once
182       uint _earmarkedShares;
183       uint _length = _redemptionContracts.length();
184       for (uint i; i < _length; ++i) {
185         _earmarkedShares +=
          ITreehouseRedemption(_redemptionContracts.at(i)).totalRedeeming();
186       }
187
188       uint _approximateEarmark = IERC4626(TASSET).convertToAssets(_earmarkedShares);
189
190       _totalRedeemable = _approximateEarmark > _underlyingInVault ? 0 :
          _underlyingInVault - _approximateEarmark;
191     }
```

But the claim it reserves for is capped at its redeem-time value:

```
170       address _underlying = VAULT.getUnderlying();
171
172       uint _gross = _getReturnAmount(_redeem.assets, _redeem.baseRate, _assets,
          _getBaseRate());
173       uint _holderFee = (_gross * redemptionFee) / PRECISION;
174       uint _treasuryCut = (_gross * treasuryFee) / PRECISION;
175       uint _returnAmount = _gross - _holderFee - _treasuryCut;
176
177       if (_gross > _redeem.assets) revert RedemptionError();
178
179       // amount actually leaving the vault: user payout + treasury cut (holderFee
          stays as donated IAU)
180       uint _outflow = _returnAmount + _treasuryCut;
181       if (IERC20(_underlying).balanceOf(address(VAULT)) < _outflow) revert
          InsufficientFundsInVault();
```

Recommendation

In TreehouseRedemptionV3, track the recorded asset value alongside the share count (rename to `totalRedeemingShares`) and reserve the greater of the two, so the earmark is never below the redeem-time cap:

```
// TreehouseRedemptionV3: maintain alongside totalRedeeming
uint public totalRedeemingAssets;  // += _assets in _processRedeem, -= _redeem.assets in
finalizeRedeem
```

And return a computed `totalRedeeming()` using `totalRedeemingAssets` and `totalRedeemingShares` :

```
function totalRedeeming() public view returns (uint) {
    uint earmarkShares = IERC4626(TASSET).convertToShares(totalRedeemingAssets);
    return totalRedeemingShares > earmarkShares ? totalRedeemingShares : earmarkShares;
}
```

Status

✓ Fixed

[WP-L6] Blacklist asymmetry: escrowed value escapes the blacklist, while blacklisting V3 freezes every queued exit

Low

Issue Description

`TAsset._update` reverts when `from`, `to`, or `msg.sender` is blacklisted. In `finalizeRedeem` the share burn is performed **by V3 itself** — `IERC4626(TASSET).redeem(_redeem.shares, address(this), address(this))` — so the redeeming user's address never appears in the checked set. The payout that follows is wstETH moved by `RedemptionController`, which has no blacklist at all.

A user blacklisted **after** queuing can still finalize and receive wstETH. The blacklist does not reach value already escrowed, so it cannot stop an exit that is already in flight.

The structure is inherited from V2 rather than introduced here, but this upgrade widens the exposed window: the minimum drops from 250 to 50 and `redeemWithNft` removes the floor entirely, so more value sits in the queue at any given time.

Impact: the blacklist control does not hold over in-flight redemptions, and its only effective use against the queue is a protocol-wide freeze.

Vulnerable Code

The checked addresses never include the redeeming user:

```
83    /**
84     * @dev Will revert blacklisted addresses
85     */
86    function _update(
87      address from,
88      address to,
89      uint value
90    ) internal virtual override notBlacklisted(from) notBlacklisted(to)
      notBlacklisted(msg.sender) {
91      super._update(from, to, value);
92    }
```

Because V3 is both the caller and the owner of the burn:

```
160     function finalizeRedeem(
161       uint _redeemIndex
162     ) external nonReentrant whenNotPaused validateRedeem(msg.sender, _redeemIndex) {
163       RedemptionInfo storage _redeem = redemptionInfo[msg.sender][_redeemIndex];
164
165       if (block.timestamp < _redeem.startTime + waitingPeriod) revert
          InWaitingPeriod();
166       uint _assets = IERC4626(TASSET).redeem(_redeem.shares, address(this),
          address(this));
167       redeeming[msg.sender] -= _redeem.shares;
168       totalRedeeming -= _redeem.shares;
```

Recommendation

Decide explicitly which behaviour is intended, and make the code say so. If escrowed value should remain subject to the blacklist, check the beneficiary at settlement:

```
function finalizeRedeem(uint _redeemIndex) external ... {
  if (IBlacklistable(TASSET).isBlacklisted(msg.sender)) revert Blacklisted();
  ...
}
```

If instead in-flight redemptions are meant to be final once queued, document that, so the blacklist is not relied on as a control over the queue during an incident.

Status

✓ Fixed

[WP-I7] Ops timelock proposer/executor set is a placeholder (admin multisig) — deploying as-is collapses the two-tier governance model

Informational

Issue Description

In `script/RedemptionUpdate.s.sol`, `run()` seeds the NEW ops `TimelockController` with `proposers = [ADMIN_MS]` and `executors = [ADMIN_MS]`, explicitly flagged `TODO(team): replace proposers/executors with the intended ops-governance set; ADMIN_MS is a PLACEHOLDER`. The artifact generator likewise uses `ADMIN_MS` as `createdFromSafeAddress` with a matching TODO. If shipped unmodified, the "1-day ops" and "5-day main" tiers are controlled by the SAME admin Safe, so the separation the accepted-risk model depends on (see the live-fees finding: "the ops-governance set below is the primary protection") does not exist — the admin Safe alone can move fees / `waitingPeriod` / the exemption-gate pointer at 1-day speed.

Impact: the stated two-tier security model is void until the placeholder is replaced; this directly undercuts the mitigation relied on for the retroactive-fee/freeze risk.

PoC

Configuration issue (not an on-chain exploit):

1. `run()` deploys the ops `TimelockController(OPS_DELAY, [ADMIN_MS], [ADMIN_MS], address(0))`.
2. The ops timelock's sole proposer/executor is the same admin Safe that proposes to the main timelock.
3. One actor controls both the 1-day ops tier and the 5-day main tier; the "separate ops governance" protection is absent.

Ops timelock construction seeded with the placeholder admin Safe:

```
143     // 1. ops timelock — owns the parameter periphery (fees / NFT config)
144     // TODO(team): replace proposers/executors with the intended ops-governance
    set;
145     //             ADMIN_MS is a PLACEHOLDER
```

```
146     address[] memory _proposers = new address[](1);
147     _proposers[0] = ADMIN_MS;
148     address[] memory _executors = new address[](1);
149     _executors[0] = ADMIN_MS;
150     TimelockController _opsTimelock = new TimelockController(
151       OPS_DELAY,
152       _proposers,
153       _executors,
154       address(0) // self-administered, no admin backdoor
155     );
```

`createdFromSafeAddress` artifact placeholder with matching TODO:

```
293     // TODO(team): createdFromSafeAddress uses ADMIN_MS as PLACEHOLDER until the
        ops
294     //             proposer set is decided
295     (_t, _v, _p) = _opsBatch(_gate, _v3);
296     _emitBatch('OPS TIMELOCK batch (1-day delay, proposer = TODO placeholder
        adminMs)', _opsTimelock, ADMIN_MS, _t, _v, _p, SALT_OPS, OPS_DELAY,
        'redemption-ops');
```

Recommendation

Before mainnet, replace the ops timelock proposers/executors with the intended ops-governance set (distinct from the main admin Safe) and set `createdFromSafeAddress` accordingly. Add a deploy-time `require` /CI assertion that the ops timelock proposer set does not equal the main admin Safe, so the two-tier model can't be shipped collapsed.

```
// ops-governance set, distinct from the main admin Safe
_proposers[0] = OPS_MS;
_executors[0] = OPS_MS;
require(OPS_MS != ADMIN_MS, "ops timelock proposer must differ from main admin Safe");
```

Status

✓ Fixed

[WP-I8] The fast-pause EOA can also un-pause, so a leaked pause key can undo an emergency stop

Informational

Issue Description

`PathPausable.setPathPause(bool)` treats `owner()` and `pauser` identically for both directions of the toggle. The `pauser` configured in the deploy script is a single ”fast pause EOA” — a hot key held for incident response — so that key can clear a pause as readily as set one.

This inverts the intent of a dedicated pause role. A fast key exists so a low-trust holder can stop the system quickly; restarting it is a decision that should require the same authority as any other state change. As written, an attacker who compromises the pause key can un-pause immediately after governance pauses in response to an incident, and can keep doing so, forcing a 5-day owner action to rotate the key before a pause can be made to stick.

This is the mirror image of the separately-reported concern that the pauser can freeze finalization of already-committed redemptions: that one is about the key having too much reach when used, this one is about it being able to remove a protection it did not apply.

```
32    /**
33     * @notice Set the path-level pause state of this contract
34     * @dev owner or pauser. Does not affect the global pause source. Idempotent: a
    call
35     *      that doesn't change state is a silent no-op, so PathPauseSet events map
    1:1
36     *      to actual state changes
37     * @param _paused is contract paused
38     */
39    function setPathPause(bool _paused) external {
40      if (msg.sender != owner() && msg.sender != pauser) revert Unauthorized();
41      if (pathPaused == _paused) return;
42      emit PathPauseSet(_paused);
43      pathPaused = _paused;
44    }
```

Recommendation

Make the pause role one-way — the pauser may engage a pause, only the owner may release it:

```
function setPathPause(bool _paused) external {
  if (msg.sender == pauser) {
    if (!_paused) revert Unauthorized();     // pauser may only pause
  } else if (msg.sender != owner()) {
    revert Unauthorized();
  }
  if (pathPaused == _paused) return;
  emit PathPauseSet(_paused);
  pathPaused = _paused;
}
```

Status

ⓘ Acknowledged

[WP-I9] Fastlane checks the liquidity reservation before the 4626 redeem, allowing a 1-wei violation

Informational

Issue Description

`redeemAndFinalize` evaluates `getRedeemableAmount()` against `previewRedeem(_shares)` **before** performing the redemption. The redemption itself moves the 4626 ratio, and the rounding residue it leaves behind can raise the remaining shares' `convertToAssets` by one wei after the check has already passed. The reservation invariant is then violated by that wei, and a pending delayed claim can revert with `InsufficientFundsInVault` .

Reaching it requires the vault to be funded to exactly the earmark plus the fastlane output, and the pending claim to have no holder-fee cushion — so it is a dust-scale edge case, reported as informational. It is noted because the ordering is trivially correctable.

```
86    function redeemAndFinalize(uint96 _shares) external nonReentrant whenNotPaused {
87      uint _assets = IERC4626(TASSET).previewRedeem(_shares);
88      if (_assets < minRedeemInUnderlying) revert MinimumNotMet();
89      if (getRedeemableAmount() < _assets) revert InsufficientFundsInVault();
90
91      IERC20(TASSET).safeTransferFrom(msg.sender, address(this), _shares);
92      _assets = IERC4626(TASSET).redeem(_shares, address(this), address(this));
```

Recommendation

Re-check the reservation against the realised amount, after the redeem has settled the new ratio:

```
IERC20(TASSET).safeTransferFrom(msg.sender, address(this), _shares);
_assets = IERC4626(TASSET).redeem(_shares, address(this), address(this));
if (getRedeemableAmount() < _assets) revert InsufficientFundsInVault();
```

Status

✓ Fixed

[WP-G10] Fastlane issues an unconditional treasury payout even when the fee rounds to zero

Gas

Issue Description

`redeemAndFinalize` always calls `REDEMPTION_CONTROLLER.redeem(_fee, treasury)`, including when `_fee == 0`. That happens whenever `defaultFee` is zeroed, a user has a 0-bip override, or the amount is small enough that `(_assets * fee) / 1e4` floors to zero. Each such call still runs the controller's whitelist check and a full `safeTransferFrom` of zero, for no effect.

`TreehouseRedemptionV3` already guards the analogous call, so the two paths are inconsistent with each other.

Impact: wasted gas on the atomic hot path. No correctness issue — wstETH permits zero-value transfers.

```
93      uint _fee = feeContract.applyFee(_assets, msg.sender);
94
95      IInternalAccountingUnit(IAU).burn(_assets);
96
97      REDEMPTION_CONTROLLER.redeem(_assets - _fee, msg.sender);
98      REDEMPTION_CONTROLLER.redeem(_fee, treasury);
99
100     emit Redeemed(msg.sender, _shares, _assets, _fee);
```

V3 guards the equivalent call:

```
183     IInternalAccountingUnit(IAU).burn(_outflow);
184     REDEMPTION_CONTROLLER.redeem(_returnAmount, msg.sender);
185     if (_treasuryCut > 0) {
186       REDEMPTION_CONTROLLER.redeem(_treasuryCut, treasury);
187     }
```

Recommendation

```
REDEMPTION_CONTROLLER.redeem(_assets - _fee, msg.sender);
if (_fee > 0) {
  REDEMPTION_CONTROLLER.redeem(_fee, treasury);
}
```

Status

✓ Fixed

[WP-G11] PRECISION should not be a mutable storage variable; declare it constant can save gas

Gas

Issue Description

In `TreehouseRedemptionV3`, `PRECISION` is declared as `uint32 PRECISION = 1e4;` — a mutable state variable with default (internal) visibility and no setter. It is effectively a constant but occupies a storage slot, so every read is an `SLOAD` rather than a compile-time inlined value. It is read in `finalizeRedeem` (the fee math) and in `setRedemptionFee` / `setTreasuryFee` (the fee-cap checks).

An unnecessary storage slot plus `SLOAD` cost on the redemption hot path. There is no correctness issue — the value is never mutated.

1. `PRECISION` is stored in a state slot at deployment.
2. Each `_holderFee` / `_treasuryCut` computation and each fee-setter cap check reads it via `SLOAD` instead of using an inlined constant.

```
53    uint32 PRECISION = 1e4;
```

```
172       uint _gross = _getReturnAmount(_redeem.assets, _redeem.baseRate, _assets,
      _getBaseRate());
173       uint _holderFee = (_gross * redemptionFee) / PRECISION;
174       uint _treasuryCut = (_gross * treasuryFee) / PRECISION;
175       uint _returnAmount = _gross - _holderFee - _treasuryCut;
```

Recommendation

Declare it `constant` so reads become compile-time constants and the storage slot is freed:

```
    uint32 internal constant PRECISION = 1e4;
```

Status

✓ Fixed

[WP-N12] setPause → setPathPause rename leaves off-chain pause tooling calling the old selector

Issue Description

This upgrade renames `PathPausable.setPause` to `setPathPause`. The new redemption contracts (`TreehouseRedemptionV3`, `TreehouseFastlaneV2`) both inherit `PathPausable` and now expose ONLY `setPathPause`; the old `setPause` name survives only on the unrelated legacy `TreehouseRedemption` and `RedemptionController`. Off-chain tooling, however, still calls the old name — e.g. `scripts/sim-redemption-upgrade.ts` calls `.setPause(true)` — so any ops runbook or emergency script that tries to pause the NEW contracts by the old selector will revert (there is no such function on those contracts, and the call resolves to a nonexistent function selector).

Impact: this is not a contract vulnerability — the on-chain code is correct. It is an interface-drift / operational hazard: the mismatch between the renamed on-chain function and the stale off-chain caller could delay an emergency pause of the new contracts at the exact moment it matters most.

PoC

1. An incident requires pausing `TreehouseRedemptionV3`.
2. An operator runs the existing pause script/runbook, which calls `setPause(true)` on the new contract (as `scripts/sim-redemption-upgrade.ts` still does).
3. The transaction reverts (function selector not found), delaying the pause; the operator must discover the rename and switch to `setPathPause` before the pause can take effect.

The renamed on-chain function — the new contracts expose only this name:

```
39    function setPathPause(bool _paused) external {
40      if (msg.sender != owner() && msg.sender != pauser) revert Unauthorized();
41      if (pathPaused == _paused) return;
42      emit PathPauseSet(_paused);
43      pathPaused = _paused;
44    }
```

The stale off-chain caller still invoking the old `setPause` selector:

```
65      await treehouseRedemption.connect(signer).setPause(true)
```

Recommendation

Update all off-chain pause tooling and ops runbooks that target the new contracts to use `setPathPause`. Grep the repo and ops scripts for `.setPause(` and repoint any call that references `TreehouseRedemptionV3` / `TreehouseFastlaneV2` to the renamed function:

```
// before — reverts against the new contracts (no such selector)
await treehouseRedemption.connect(signer).setPause(true)

// after — matches the renamed PathPausable function
await treehouseRedemption.connect(signer).setPathPause(true)
```

Status

ⓘ Acknowledged

Appendix

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by WatchPug; however, WatchPug does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication.

Disclaimer

This report is based on the scope of materials and documentation provided for a limited review at the time provided. Results may not be complete nor inclusive of all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Smart Contract technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. A report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.